# 1. 開発目的：大規模量子回路を実行したい

## 分散量子回路シミュレータの開発

Deferred Hierarchical Distributed State-Vector Simulator（DH-SVS、仮称）

### 容量の壁：dense 状態ベクトル

$$S(n) = 16 \times 2^n \text{ bytes}$$

`complex128`、状態 1 コピーの容量

| 量子ビット数 | 状態容量 |
|---|---|
| 32 | 64 GiB |
| 40 | 16 TiB |
| 50 | 16 PiB |

作業 Block、通信バッファ、Adjoint 状態、Checkpoint、ログなどは別途必要。

### 目標：容量と実行時間を両立する

- 単一ノード RAM を超える状態を扱う。
- RAM・SSD/HDD・MPI を統合する。
- 全状態走査と転送の繰り返しを減らす。

### 評価対象は End-to-End 時間

RAM 帯域 / ディスク I/O / MPI 通信 / 同期待ち / 振幅演算 / 計画の費用

**保存できる容量**と**時間内に計算できる規模**を分けて評価する。

本資料は実装前の設計説明。高速化効果・研究上の新規性は今後検証する。

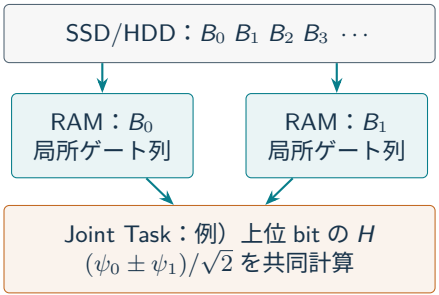
# 2. 基本戦略：状態ベクトルを Block に分割する

**振幅区間を管理・実行の単位にする**

$$|\Psi\rangle = \sum_{h=0}^{2^{n-b}-1} |h\rangle \otimes |\psi_h\rangle\,, \quad |\psi_h\rangle \in \mathbb{C}^{2^b}$$

- $b$：Block 内のインデックスビット数。
- Block-local 演算は、RAM 滞在中にまとめて実行。
- Block 間の振幅混合は、必要な Block 群の **Joint Task** で処理。

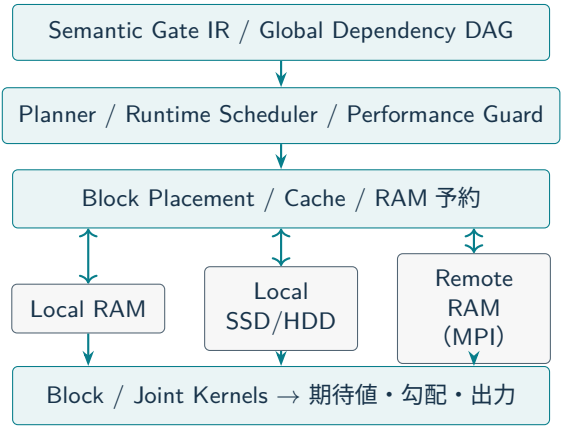
Block は独立した量子部分系ではない。未正規化の振幅と Block 間の相対位相を保つ。




Global DAG で入力版・依存順序を保証

Block を読み込むたびに**複数の演算を進める**ことで、再読み込み・書き戻しを減らす。

# 3. MPI + RAM + SSD/HDD を同じ配置モデルで扱う




**必要な Block の正しい版を取得**

1. Local RAM を再利用。
2. Local SSD/HDD から読む。
3. MPI で Remote RAM から受信。
4. 同じ base 版＋ Log から再生。

**取得経路の選択条件**

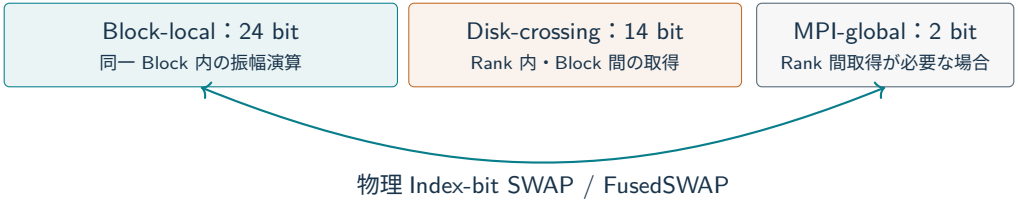
Block 版・依存 frontier・Frame・Layout が整合する候補だけを、移動／再生の費用で比較する。

Checkpoint は振幅だけでなく、版・Log・Frame・Layout の整合した情報を保存する。

Out-of-core と MPI 分散を**「必要な振幅を計算場所へ集める」**問題として統合する。

# 4. Qubit Layout：再配置の費用を後続ゲートで回収する

例：40 qubit、4 MPI rank（$g = 2$）、Block 内 $b = 24$。領域は現在の物理配置で決まる。




## 2 種類の SWAP を区別する

- **論理 SWAP**：入力回路の作用。後続 wire 対応の更新で表せる場合がある。
- **物理再配置**：論理状態を保ち、振幅配列と Layout を一緒に変更する。

## 先読み窓の総費用で選択する

$$T_{\mathrm{move}} + T_{\mathrm{after}}(W) + T_{\mathrm{align}} < T_{\mathrm{direct}}(W)$$

$W$：同じ先読み窓。移動＋窓内全ゲート＋整合の費用を、現 Layout での窓全体と比較。必要な復元費用も加える。

FusedSWAP は複数の配置変更をまとめる候補。効果は転送量と実時間で測る。

**MPI 削減量だけでなく SSD/HDD の増分 I/O も含めて**、局所化の損益を判断する。

# 5. 遅延 Log と Clifford Frame：必要時にまとめて具体化

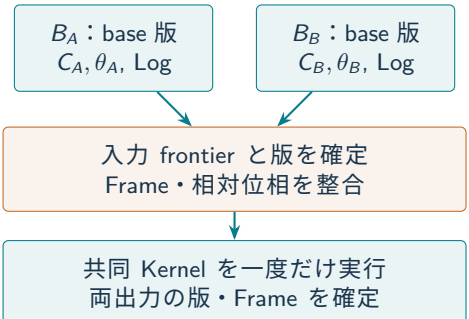
**保存振幅と論理状態を分ける**

$$|\psi_h\rangle = e^{i\theta_h} C_h |\phi_h\rangle$$

整合した論理 cut での表現。$\phi_h$：保存振幅、$C_h$：Block 内 Clifford、$\theta_h$：相対位相。

- Per-Block Log：局所演算の履歴。
- Global DAG：Block 間依存と、一意な Joint Task を保持。
- Frame Group：共通 Frame を共有し、メタデータ増大を抑える。

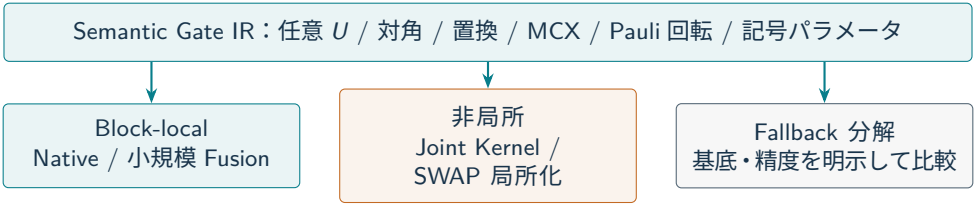
frontier ＝論理的にどこまで演算を適用したか。保存振幅の base 版とは別に管理する。




例：上位 bit の $H$ は Block 振幅を混合する

遅延実行で走査をまとめる。**全振幅の具体化コストや状態の情報量は消えない。**

# 6. Native-first：ゲートの意味と構造から実行方法を選ぶ




**専用 Kernel を優先候補にする**

- 1Q/2Q 小密行列、対角・置換。
- MCX：X / CNOT / CCX / CCCX 以上を条件付き振幅置換として統一。
- 任意 $U$ を検証して受付。実行幅・行列容量の上限は実測で決める。

**パラメータ変更で計画を再利用**

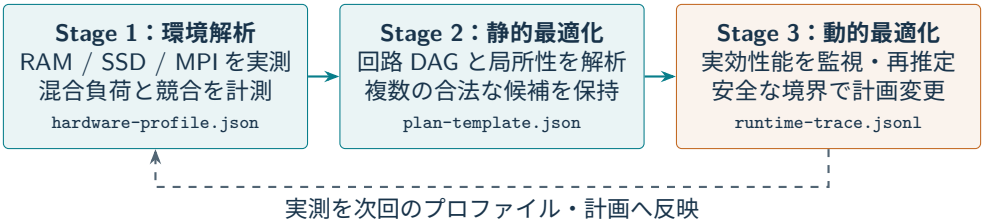
$$R_z(\alpha)R_z(\beta) = R_z(\alpha+\beta)$$

記号式を融合 → 値を bind → 値依存候補を guard 付きで特殊化。

prefix 状態の再利用は、依存パラメータ・入力状態・frontier・Frame・Layout が一致する場合のみ。

Native・Fusion・Joint・SWAP・分解を、**精度とデータ移動を含む総費用**で比較する。

# 7. 3段階の最適化：計測→計画→実行時適応




**Performance Guard：劣化を検知**

- 同じ I/O サイズ・キュー深度・通信形・並行負荷の想定値と比較。
- 移動平均・継続時間・ヒステリシスで一時的な揺らぎを除く。

**効果と切替費用を比較して変更**

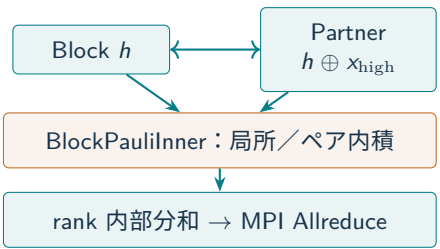
- 先読み深さ・並列度・取得元は Task 境界で調整。
- Layout 等は版・DAG・Frame が整合する **safe frontier** で確定。

残りの実行時間の短縮が、**監視・再計画・移行の追加費用**を上回るときに切り替える。

# 8. 期待値・Backpropagation も Block 単位で実行する

## Pauli 期待値：部分和＋ MPI 集約

$$E = \langle \Psi | O | \Psi \rangle, \quad O = \sum_{\ell} c_\ell P_\ell$$




$x_{\mathrm{high}}$：Pauli の X/Y が反転する上位 bit の mask。全状態を 1 ノードに Gather せず計算する。

## Adjoint：2 状態を逆方向に伝播

$$\psi_j = U_j \psi_{j-1}, \qquad \lambda_m = O\psi_m, \quad O = O^\dagger$$

$O$ は $\theta_j$ に非依存。$\lambda_{j-1} = U_j^\dagger \lambda_j$。Forward も $U_j^\dagger$ で逆再生。

$$\frac{\partial E}{\partial \theta_j} = 2\,\mathrm{Re}\,\langle \lambda_j | \frac{\partial U_j}{\partial \theta_j} | \psi_{j-1} \rangle$$

- 非局所の逆ゲートは両状態で通信が必要。
- 基本容量は $2S(n)$。40 qubit で **32 TiB** ＋作業領域。
- 逆再生・Checkpoint/再計算・Parameter-shift を I/O 込みで比較。

対象は途中測定のないユニタリー回路。内積の frontier・Frame・Layout を整合させる。

Pauli 作用・Partner 取得・内積を共通化し、**VQE の反復と勾配評価全体**を最適化する。

# 9. Resource Estimator：実行前に容量と時間を見積もる

**Hardware Profile ＋回路＋ Plan 候補**

- RAM ピーク、ストレージ、ノード数。
- SSD/HDD read/write、MPI 送受信量。
- 全状態走査換算、Checkpoint 容量。
- 予想時間、ボトルネック、残時間。

**転送費用の基本モデル**

$T_{\mathrm{fetch}} \approx L + B/\beta_{\mathrm{eff}}$
$L$：遅延、$B$：バイト数、$\beta_{\mathrm{eff}}$：実効帯域。

並行処理の重なりと競合は混合負荷計測・DAG で評価する。全 Task の費用の単純和にはしない。

**取得候補の比較例（説明用の仮定値）**
同じ 256 MiB Block の正しい版が取得可能、遅延を省略、処理の重なりなしと仮定。

| 経路 | 仮定 | 費用 |
|---|---|---|
| Local SSD | 2 GiB/s | 125 ms |
| Remote RAM | 8 GiB/s | 31 ms |
| Log 再生 | 40 ms | 40 ms |

Log 再生は**同じ base が RAM にある**場合。base 取得・ログ転送が必要ならその費用も加算。

MPI が 1 GiB/s へ低下すると 250 ms
同じ Block でも、実効性能の変化で選ぶ経路が変わる。

見積りに**前提と誤差**を添え、実行 trace で校正する。例の数値は性能測定結果ではない。

# 10. 開発ロードマップ：意味の正しさから実機評価へ

設計要素は初期から型・不変条件に含め、実行 Backend と最適化を段階的に拡張する。

| 段階 | 主な実装 | 次へ進むための確認 |
|---|---|---|
| 1 | NumPy 参照・意味モデル | 8〜12 qubit で DAG / Log / Frame / Layout、任意 $U$・MCX・bind の数値一致。 |
| 2 | 単一ノード RAM ＋ SSD | Block I/O、遅延具体化、Frame・相対位相、Checkpoint 復旧を検証。 |
| 3 | MPI ＋ SSD | Joint / SWAP / 取得経路の版整合。1 / 2 / 4 rank で照合。 |
| 4 | 期待値・勾配・反復 | 参照期待値・有限差分勾配と照合。prefix 再利用と追加 I/O を検証。 |
| 5 | Guard・Estimator | 帯域低下注入で再計画を検証。32〜36 qubit 実測から 40 qubit 級の容量・時間を推定。 |

**比較対象を揃える**
通常の Block batching / deferred-no-frame / Frame 共有 / SWAP なし・あり / 固定 Plan・適応 Plan。

**記録する指標**
数値誤差、総時間、I/O・MPI バイト、Cache ヒット、移行・再計画費、再開性、推定誤差。

最初の成果物：**不変条件の ADR ＋小規模参照一致＋版付き trace**。その上で改善量を測る。

根拠資料：`ref/`内の開発仕様・スライド構成案。各フレームの TeX 内 note に参照節を記載。